Metodologie di Programmazione (A-L)

Il semestre - a.a. 2023 – 2024

Parte 9 – Tipi Generici**

a cura di Mattia Samory*



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

📣Avvisi

- ci sarà lezione venerdì 26/4 e lunedì 29/4

Letture

- Tipi generici (Horstmann 18)

🔄Recap

- classi interne, anonime, espressioni lambda

Che cosa accomuna queste due classi?

```java
public class CoppiaDiInteri
{
    private int a, b;

    public CoppiaDiInteri(int a, int b)
    {
        this.a = a;
        this.b = b;
    }

    public int getPrimo() { return a; }
    public int getSecondo() { return b; }
}
```

```java
public class CoppiaDiDouble
{
    private double a, b;

    public CoppiaDiDouble(double a, double b)
    {
        this.a = a;
        this.b = b;
    }

    public double getPrimo() { return a; }
    public double getSecondo() { return b; }
}
```



Che cos'è un generico?

- I tipi generici, in Java, sono un modello di programmazione che permette di definire, con una sola dichiarazione, un intero insieme di metodi o di classi
- Un meccanismo **MOLTO** potente


Cup<T>

Avete già utilizzato ampiamente i generici!

- In tutte le collezioni Java!
- Creare istanze di classi con generici:

```
new ArrayList<String>();
```

- Dichiarare e assegnare variabili di tipi generici

```
List<String> listaDiStringhe = new ArrayList<String>();
```

- Dichiarare metodi che prendono in input tipi generici

```
public void metodo(List<String> lista)
{
    // ...
}
```

Esempio di classe generica

```
public class Valore<T>
{
    private final T val;

    public Valore(T val) { this.val = val; }

    public T get() { return val; }

    @Override
    public String toString() { return ""+val; }

    public String getType() { return val.getClass().getName(); }
}
```

Definisce un tipo generico della classe

Usa il tipo generico della classe

- Per definire un tipo generico della classe, si utilizza la sintassi a parentesi angolari dopo il nome della classe con il tipo generico da utilizzare
- Da quel punto, si utilizza il tipo generico come un qualsiasi altro tipo di classe

Istanziare la classe generica

Inferenza automatica del tipo generico

```
public static void main(String[] args)
{
    Valore<Integer> i = new Valore<>(42);
    Valore<String> s = new Valore<>("ciao");
    Valore<Valore<String>> v = new Valore<>(s);

    System.out.println(i.get()+":"+i.getType());
    System.out.println(s.get()+":"+s.getType());
    System.out.println(v.get()+":"+v.getType());
}
```

- L'output sarà:

```
42:java.lang.Integer
ciao:java.lang.String
ciao:Valore
```

Un esempio concreto: una classe Grafo

- Il tipo del vertice può essere qualsiasi
- Utilizzare Object fa perdere ogni informazione sul tipo del vertice e costringerebbe a continui **downcast**

```
public class Grafo<V>
{
    private Set<V> vertici;
    /* archi da implementare */

    public void addVertex(V v) { /* ... */ }
    public V getVertex(String id)
    {
        for (V v : vertici)
            if (v.toString().equals(id)) return v;
        return null;
    }
}
```

Altro esempio: Coppia di elementi di tipo generico

```
public class Coppia<T>
{
    private T a, b;

    public Coppia(T a, T b)
    {
        this.a = a;
        this.b = b;
    }

    public T getPrimo() { return a; }
    public T getSecondo() { return b; }
}
```

Esempio: utilizzare la classe Coppia

- Semplicemente si istanzia la classe specificando il tipo desiderato:

```
Coppia<Integer> ci = new Coppia<Integer>(10, 20);
Coppia<Double> cd = new Coppia<Double>(10.5, 20.5);
Coppia<String> cs = new Coppia<String>("abc", "def");
Coppia<Object> co = new Coppia<Object>("abc", 20);
```

- …e chi più ne ha più ne metta!

Specificare più tipi generici di classe

- Specifichiamo i tipi generici separati da virgola:

```
public class Coppia<T, S>
{
    private T a;
    private S b;

    public Coppia(T a, S b)
    {
        this.a = a;
        this.b = b;
    }

    public T getPrimo() { return a; }
    public S getSecondo() { return b; }
}
```

- Per convenzione i tipi generici sono chiamati con le lettere **T**, **S**, ecc. (**E** nel caso in cui siano elementi di una collection)

I generici funzionano solo con i tipi derivati

- Non è possibile utilizzare tipi primitivi, ad es. int, double, char, ecc.
- Es:

```
Valore<int> v;
Valore<double> v;
```

- Versioni future di Java potrebbero permetterlo

I tipi generici differiscono sulla base dei loro tipi

- Ad esempio, Valore<Integer> non è compatibile con Valore<String>:

```
public static void main(String[] args)
{
    Valore<Integer> i = new Valore<>(42);
    Valore<String> s = new Valore<>("ciao");
    Valore<Valore<String>> v = new Valore<>(s);

    s = i;

    Syst
    Syst
    Syst
}
```

Type mismatch: cannot convert from Valore<Integer> to Valore<String>

2 quick fixes available:

- Change type of 's' to 'Valore<Integer>'
- Change type of 'i' to 'Valore<String>'

Press 'F2' for focus

Estendere le classi generiche

- Ovviamente è possibile estendere le classi generiche per creare classi più specifiche
- Ad esempio, una classe Orario può estendere la classe Coppia:

```
public class Orario extends Coppia<Integer, Integer>
{
    public Orario(Integer a, Integer b)
    {
        super(a, b);
    }
}
```

- O una classe Data:

```
public class Data extends Coppia<Integer, Coppia<Integer, Integer>>
{
    public Data(Integer giorno, Integer mese, Integer anno)
    {
        super(giorno, new Coppia<Integer, Integer>(mese, anno));
    }
}
```



Esercizio: Lista Linkata generica

- Progettare una classe generica ListaLinkata

```
public class ListaLinkata<T>
{
    private Elemento first;

    private class Elemento
    {
        private T val;
        private Elemento next;
        public Elemento(T val, Elemento next) { this.val = val; this.next = next; }
    }

    public void addFirst(T e)
    {
        first = new Elemento(e, first);
    }

    public void removeFirst()
    {
        first = first.next;
    }

    public String toString()
    {
        StringBuffer sb = new StringBuffer();
        Elemento e = first;
        while(e != null) { sb.append(e.val); if (e.next != null) sb.append(","); e = e.next; }
        return sb.toString();
    }
}
```



Esercizio: la classe Pila

- Progettare una classe generica Pila implementata mediante lista di elementi (provate anche con l'array)
- La classe è costruita con la dimensione iniziale dell'array ed implementa i seguenti metodi:
  - push: inserisce un elemento in cima alla pila
  - pop: elimina e restituisce l'elemento in cima alla pila
  - peek: restituisce l'elemento in cima alla pila
  - isEmpty: restituisce true se la pila è vuota

Le interfacce Comparable e Comparator sono generiche

```
public interface Comparable<T>
{
        int compareTo(T o);
}

public interface Comparator<T>
{
        int compare(T o1, T o2);
}
```

Esempio: estendere un'interfaccia generica con vincolo di comparabilità sul tipo generico

```
public interface MinMax<T extends Comparable<T>>
{
        T min();
        T max();
}

public class MyClass<T extends Comparable<T>>
                        implements MinMax<T>
{
        // …
}
```

Quali di queste estensioni sono corrette?

```
class MyClass<T extends Comparable<T>> implements
MinMax<T extends Comparable<T>> { /* … */ }
```



```
class MyClass implements MinMax<T> { /* … */ }
```



```
class MyClass implements MinMax<Integer> { /* … */ }
```



Generici e collezioni (1)

- Alcuni esempi prototipici:

```
public interface List<E>
{
    void add(E x);
    Iterator<E> iterator();
}
public interface Iterator<E>
{
    E next();
    boolean hasNext();
    default void remove();
}
```

Generici e collezioni (2)

- L'utilizzo principale dei tipi generici è nelle collezioni
- Ad esempio, vediamo come è definita ArrayList:

```
public class ArrayList<E>
extends AbstractList<E>
implements List<E>, RandomAccess, Cloneable, Serializable
```

- Quando scriviamo:

```
new ArrayList<String>();
```

- La classe viene trattata come: public class ArrayList<String> extends AbstractList<String>

Definire un metodo generico

- Per definire un **metodo** generico **con proprio tipo generico** è necessario anteporre il tipo generico tra parentesi angolari al tipo di ritorno:

```
static public <T> void esamina(ArrayList<T> lista)
{
    for (T o : lista)
        System.out.println(o.toString());
}
```

Perché posso chiamare toString() su un tipo generico?



Esercizio: Inverti lista e massimo

- Implementare i seguenti due metodi generici statici:
  - inverti, che data in input una lista di elementi di tipo generico, restituisca un'altra lista con gli elementi in ordine invertito
  - max, che data in input una lista di elementi di tipo generico, ne restituisca il valore massimo
- **Nota:** per il secondo metodo è necessario utilizzare il costrutto <T extends InterfacciaOClasse>, che impone un vincolo sul supertipo di T

Trova le differenze

```
static public void esamina(ArrayList<Frutto> frutti)
{
    // ...
}
```

Accetta un ArrayList<Arancia>? Perché?

```
static public <T extends Frutto> void esamina(ArrayList<T> frutti)
{
    // ...
}
```

Indica che **T** può essere di tipo Frutto **OPPURE** un suo sottotipo

Per le classi generiche non vale l'ereditarietà dei tipi generici

- Ad esempio, ArrayList<Integer> non è di tipo ArrayList<Number> o ArrayList<Object>:

```
ArrayList<Number> listaDiNumeri
                = new ArrayList<Integer>();
```

Type mismatch: cannot convert from ArrayList<Integer> to ArrayList<Number>

1 quick fix available:

Change type of 'listaDiNumeri' to 'ArrayList<Integer>'

Press 'F2' for focus

- Ma rimane comunque l'ereditarietà tra classi:

```
List<Integer> listaDiNumeri
                = new ArrayList<Integer>();
```

Per rendere “sicura” la collection a tempo di compilazione (1)

- Se fosse permesso fare l’upcasting tra tipi generici si avrebbero situazioni imprevedibili:

```
public class ViolenzaSuUnArrayList
{
    public static void main(String[] args)
    {
        ArrayList<Mela> mele = new ArrayList<Mela>();

        prendiFrutta(mele);

        // qui avrei una lista di mele con una pera!!!
        System.out.println(mele.toString());
    }

    public static void prendiFrutta(ArrayList<Frutto> frutti)
    {
        frutti.add(new Pera());
    }
}
```

- Il controllo di consistenza di tipo viene effettuato solo a tempo di compilazione

Per rendere “sicura” la collection a tempo di compilazione (2)

- E’ possibile permettere il passaggio di sottotipi di Frutto utilizzando la sintassi **T** extends Frutto:

```
public class ViolenzaSuUnArrayList
{
    public static void main(String[] args)
    {
        ArrayList<Mela> mele = new ArrayList<Mela>();

        prendiFrutta(mele);

        // qui avrei una lista di mele con una pera!!!
        System.out.println(mele.toString());
    }

    public static <T extends Frutto> void prendiFrutta(ArrayList<T> frutti)
    {
        frutti.add(new Pera());
    }
}
```

- Tuttavia non è possibile “aggiungere” una pera a un elenco di mele…

Differenze tra array e collection

- L'upcasting è invece possibile ***a tempo di compilazione*** con gli array:

```
public class ViolenzaSuUnArray
{
    public static void main(String[] args)
    {
        Mela[] mele = new Mela[] { new Mela(), new Mela() };

        prendiFrutta(mele);

        // qui ho un array di mele con una pera!!!
        System.out.println(Arrays.toString(mele));
    }

    public static void prendiFrutta(Frutto[] frutti)
    {
        frutti[1] = new Pera();
    }
}
```

- A tempo di esecuzione otteniamo però un'**eccezione** se violiamo il contratto:

```
Exception in thread "main" java.lang.ArrayStoreException: Pera
        at ViolenzaSuUnArray.prendiFrutta(ViolenzaSuUnArray.java:17)
        at ViolenzaSuUnArray.main(ViolenzaSuUnArray.java:9)
```

Jolly come tipi generici

- Nel caso in cui non sia necessario utilizzare il tipo generico **T** nel corpo della classe o del metodo, è possibile utilizzare il jolly ?

```
public static void mangia(ArrayList<? extends Mangiabile> frutta)
{
    // mangia la frutta
}
```

- Equivalente a:

```
public static <T extends Mangiabile> void mangia2(ArrayList<T> frutta)
{
    // mangia la frutta
}
```

- ma senza la nozione del tipo utilizzato

Usare <?> per prendere in input un oggetto di una classe con qualsiasi tipo parametrico

- Nel caso in cui non sia necessario conoscere il tipo parametrico, si può utilizzare <?>
- Ad esempio:

```
public class Punto<T extends Number>
{
    private T x;
    private T y;

    public Punto(T x, T y) { this.x = x; this.y = y; }

    @Override
    public String toString() { return "("+x+";"+y+")"; }

    public static void main(String[] args)
    {
        Punto<?> p = new Punto<Integer>(10, 42);
        System.out.println(p);

        p = new Punto<Double>(11.0, 43.5);
        System.out.println(p);
    }
}
```



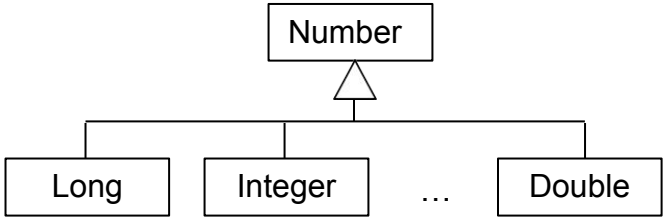
Esercizio: Successioni numeriche

- Scrivere una classe SuccessioniNumeriche, i cui oggetti sono inizialmente vuoti
- La classe espone un metodo addSuccessione che, preso in input un nome e una sequenza numerica, consente di aggiungere una successione (di lunghezza finita) del tipo numerico generico specificato (Integer, Float, Long, ecc.)
- La classe ha inoltre un metodo getSuccessione(String nome) con cui è possibile recuperare una successione a partire dal nome mnemonico
- Prevedere le seguenti successioni:
  - «Fibonacci»: 1, 1, 2, 3, 5, 8, 13, 21, 34
  - «1/n»: 1, 1/2, 1/3, 1/4, 1/5, 1/6
  - «RandomLong»: contiene oggetti Long costruiti casualmente
- **Hint:** utilizzare la classe Number come superclasse del tipo generico



Esempio: metodo generico di somma (1)

- Implementare un metodo generico somma che calcoli la somma di tutti i numeri contenuti in una collezione
- Implementazione **non generica**:




```
public class SommaNumeri
{
    public static void main(String[] args)
    {
        // interi e double
        Number[] numeri = { 1, 1.5, 2, 2.5 };
        ArrayList<Number> listaDiNumeri = new ArrayList<Number>();

        for (Number n : numeri)
            listaDiNumeri.add(n);

        System.out.printf("La lista contiene: %s\n", listaDiNumeri);
        System.out.printf("La somma dei numeri nella lista è: %.1f\n", somma(listaDiNumeri));
    }

    public static double somma(ArrayList<Number> lista)
    {
        double totale = 0.0;

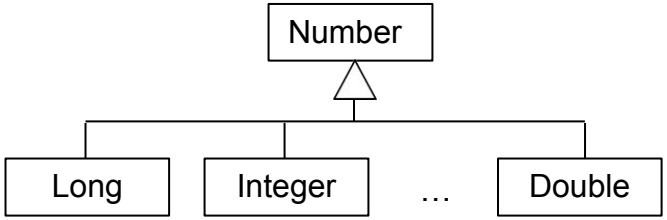
        for (Number n : lista)
            totale += n.doubleValue();

        return totale;
    }
}
```



Esempio: metodo generico di somma (1)

- Implementare un metodo generico somma che calcoli la somma di tutti i numeri contenuti in una collezione
- Implementazione **non generica**
- Posso creare ArrayList<Number> contenenti interi, double, ecc. **misti**



- Tuttavia, non posso passare in input un ArrayList<Integer>:

```
public static void main(String[] args)
{
    // interi
    Integer[] numeri = { 1, 2, 3, 4 };
    ArrayList<Integer> listaDiNumeri = new ArrayList<Integer>();

    for (Integer n : numeri)
        listaDiNumeri.add(n);

    System.out.printf("La lista contiene: %s\n", listaDiNumeri);
    System.out.printf("La somma dei numeri nella lista è: %.1f\n", somma(listaDiNumeri));
}
```



Esempio: metodo generico di somma (2)

- Implementare un metodo generico somma che calcoli la somma di tutti i numeri contenuti in una collezione
- Implementazione **generica**:

```
public class SommaNumeriGenerico
{
    public static void main(String[] args)
    {
        // interi
        Integer[] numeri = { 1, 2, 3, 4 };
        ArrayList<Integer> listaDiNumeri = new ArrayList<Integer>();

        for (Integer n : numeri)
            listaDiNumeri.add(n);

        System.out.printf("La lista contiene: %s\n", listaDiNumeri);
        System.out.printf("La somma dei numeri nella lista è: %.1f\n", somma(listaDiNumeri));
    }

    public static double somma(ArrayList<? extends Number> lista)
    {
        double totale = 0.0;

        for (Number n : lista)
            totale += n.doubleValue();

        return totale;
    }
}
```

Come funziona dietro le quinte?

- Mediante la cancellazione del tipo (type erasure)



- Quando il compilatore traduce il metodo/la classe generica in bytecode Java:

1. Elimina la sezione del tipo parametrico e sostituisce il tipo parametrico con quello reale
2. Per default il tipo generico viene sostituito con il tipo Object (a meno di vincoli sul tipo)

- Solo una copia del metodo o della classe viene creata!

Cancellazione del tipo: la classe Coppia

```java
public class Coppia<T, S>
{
    private T a;
    private S b;

    public Coppia(T a, S b)
    {
        this.a = a;
        this.b = b;
    }

    public T getPrimo() { return a; }
    public S getSecondo() { return b; }
}
```



```java
public class Coppia
{
    private Object a;
    private Object b;

    public Coppia(Object a, Object b)
    {
        this.a = a;
        this.b = b;
    }

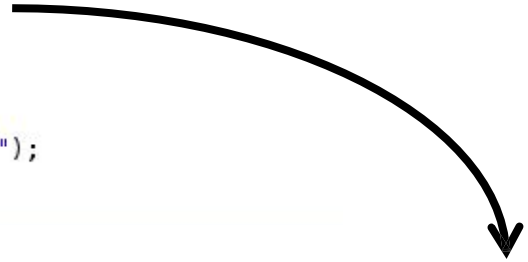
    public Object getPrimo() { return a; }
    public Object getSecondo() { return b; }

}
```

Cancellazione del tipo: il metodo massimo

```java
public class Massimo
{
    static public <T extends Comparable<T>> T getMassimo(T a, T b, T c)
    {
        if (a.compareTo(b) > 0) return a.compareTo(c) >= 0 ? a : c;
        else return b.compareTo(c) >= 0 ? b : c;
    }

    public static void main(String[] args)
    {
        int max = getMassimo(10, 20, 30);
        String s = getMassimo("abc", "def", "ghi");
    }
}
```



```java
public class Massimo
{
    static public Comparable getMassimo(Comparable a, Comparable b, Comparable c)
    {
        if (a.compareTo(b) > 0) return a.compareTo(c) >= 0 ? a : c;
        else return b.compareTo(c) >= 0 ? b : c;
    }

    public static void main(String[] args)
    {
        int max = (Integer)getMassimo(10, 20, 30);
        String s = (String)getMassimo("abc", "def", "ghi");
    }
}
```



Esercizio: cancellazione del tipo nella classe Pila

- Mostrare come viene cancellato il tipo dal compilatore nella classe Pila creata in un precedente esercizio
- Mostrare inoltre come viene trasformato il seguente codice:

```
Pila<Integer> p = new Pila<Integer>(10);
p.push(1);
p.push(2);
Integer k = p.pop();
```

Come ottenere informazioni sull'istanza di un generico?

- Per via della cancellazione del tipo, non possiamo conoscere il tipo generico a tempo di esecuzione

```
public static void main(String[] args)
{
    List<Integer> x = Arrays.asList(4, 2);
    boolean b = x instanceof List<Integer>;
}
```

Cannot perform instanceof check against parameterized type List<Integer>. Use the form List<?> instead since further generic type information will be erased at runtime

Press 'F2' for focus

- Ma possiamo comunque verificarne il tipo usando il wildcard ?

```
public static void main(String[] args)
{
    List<Integer> x = Arrays.asList(4, 2);
    boolean b = x instanceof List<?>; // true
}
```

Le parole chiave extends e super nei generici

```
static public void esamina(ArrayList<Frutto> frutti)
{
    // ...
}
```

- Si può imporre un vincolo sul tipo generico T mediante la parola chiave:
  - extends: T deve essere un sottotipo della classe specificata o la classe stessa (**covarianza**)
  - super: T deve essere una superclasse della classe specificata o la classe stessa (**controvarianza**)

Interpretazione del vincolo extends

Permette di imporre il vincolo sul supertipo:

```
List<? extends Number> l1 = new ArrayList<Number>();
List<? extends Number> l2 = new ArrayList<Integer>();
List<? extends Number> l3 = new ArrayList<Double>();
```

- Non posso sapere a priori quali saranno i tipi "omogenei" di l1, l2 o l3, ma per certo so che saranno di tipo Number
- Non posso quindi assumere che l2 o l3 siano interi o double, perché il tipo parametrico di l2 o l3 non lo specifica

Interpretazione del vincolo super

Permette di imporre il vincolo sul sottotipo:

```
List<? super Integer> l1 = new ArrayList<Number>();
List<? super Integer> l2 = new ArrayList<Integer>();
List<? super Integer> l3 = new ArrayList<Object>();
```

- Non posso sapere a priori quali saranno i tipi nella lista, per cui posso solo assumere che saranno certamente Object

PECS: "Producer Extends, Consumer Supers"

- extends e super esistono per due necessità primarie:
  - Leggere da/scrivere in una collezione generica
- Ci sono 3 modi:
  - List<?> lista = new ArrayList<Number>();
  - List<? extends Number> = new ArrayList<Number>();
  - List<? super Number> = new ArrayList<Number>();
- **<?>**: non so nulla sul tipo, quindi posso solo leggere, non scrivere
- **extends**: Se hai bisogno di una lista in input che "produca" valori di T, ma non puoi aggiungere elementi a questa lista
- **super**: Se hai bisogno di una lista che consumi elementi di tipo T, per scrivere nella lista, ma non puoi assumere il tipo degli stessi

Esempio: copia da una lista a un'altra

- Il modo più generale possibile di copiare con i generici da una lista di elementi di tipo T a un'altra lista è il seguente:

```
public <T> void copy(List<? extends T> src, List<? super T> dest)
{
    for (T k : src)
        dest.add(k);
}
```



Esempio di utilizzo della parola chiave super

- Implementare un metodo statico per l'ordinamento di liste utilizzando il metodo statico per l'ordinamento degli array

```
public static <T extends Comparable<? super T>> void sort(List<T> list)
{
    Object a[] = list.toArray();
    Arrays.sort(a);

    list.clear();
    for (Object o : a) list.add((T)o);
}
```

A volte “super” nei generici è necessario…



- Ma perché non posso scrivere semplicemente <T extends Comparable<T>>?
- Immaginiamo questa situazione:
  - public class Frutto implements Comparable<Frutto>
  - public class Pera extends Frutto implements Comparable<Pera> **non si può fare!**
    - Perché non si può implementare due volte la stessa interfaccia
  - public class Pera extends Frutto **sì**
  - Se volessi ordinare una collezione di Pera non potrei, perché Pera non estende Comparable<Pera>, ma Comparable<Frutto>

Overloading dei metodi generici

- Un metodo generico può essere sovraccaricato, come ogni altro metodo
- Anche da un metodo non generico **con lo stesso nome e numero di parametri**
- Quando il compilatore traduce una chiamata di metodo cerca il metodo più specifico
  - **Prima** il **non** generico e **poi**, eventualmente, il metodo generico

Esempio di overloading dei metodi generici

```
public class Massimo
{
    static public <T extends Comparable<T>> T getMassimo(T a, T b, T c)
    {
        if (a.compareTo(b) > 0) return a.compareTo(c) >= 0 ? a : c;
        else return b.compareTo(c) >= 0 ? b : c;
    }

    static public String getMassimo(String a, String b, String c)
    {
        // implementazione specifica per la stringa
        // ...

    }

    public static void main(String[] args)
    {
        // chiama l'implementazione generica
        int max = getMassimo(10, 20, 30);
        // chiama l'implementazione specifica per le stringhe
        String s = getMassimo("abc", "def", "ghi");
    }
}
```

I tipi raw

- I tipi generici sono stati introdotti con Java 5
- Per retrocompatibilità, è possibile istanziare una classe generica senza specificare il tipo parametrico

```
// istanza con tipo PARAMETRIZZATO
Pila<Integer> p1 = new Pila<Integer>(10);

// istanza con tipo RAW
Pila p2 = new Pila(10);
```

- E' possibile assegnare un'istanza con tipo parametrico a una con tipo raw:

```
// è possibile assegnare un'istanza
// con tipo parametrico a una con tipo raw
p2 = p1;
```

- E viceversa:

```
// e viceversa
p1 = p2;
```



Esercizio: multimappa generica

- Una multimappa è una mappa che ammette più valori a fronte di una data chiave
- Creare una classe MultiMappa generica sul tipo di chiavi e valori
- La classe implementa i seguenti metodi:
  - put(k, v) che associa il valore alla chiave specificata
  - putAll(MultiMappa) che aggiunge tutti gli elementi della multimappa in input alla mappa corrente
  - removeAll(MultiMappa) che rimuove tutte le chiavi della multimappa in input dalla mappa corrente
  - get(k) che restituisce l'insieme dei valori associati alla chiave
  - get(k, p), come sopra ma restituisce solo i valori che soddisfano il predicato p
  - values() che restituisce l'elenco (con duplicati) dei valori contenuti nella multimappa
  - valueSet() che restituisce l'insieme dei valori contenuti nella multimappa
  - transfomToMultiMappa che restituisce una multimappa in cui le coppie (k, v) sono trasformate in (k, z) secondo una funzione (k, v) -> z (z può essere di tipo diverso rispetto a quello di v)
  - mapEach che sostituisce ciascun valore v con un valore dello stesso tipo secondo una funzione (k, v) -> v'
  - la classe è iterabile sulle coppie (k, v) mediante una classe interna Elemento

Credits

Le slide di questo corso sono basate su quelle utilizzate dal Prof. Faralli, le quali sono a loro volta una rielaborazione delle slide del Prof. Navigli, e frutto della revisione degli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica Mario Marra e Paolo Straniero.